

Tutorial - 2

VISHAL A C 24CSR348

Develop a web application using React.js, Node.js, Express.js, and MongoDB to manage student attendance. Create a React components that interacts with a REST API to perform the following tasks:

- Fetch the attendance list from the server using a GET request
- Update the attendance status of a student using a PUT request
- Delete incorrect attendance records using a DELETE request
- Display the updated attendance list instantly without refreshing the page

The screenshot displays a web application titled "Student Attendance Management". At the top, there is a dark blue header bar. Below the header, the title "Student Attendance Management" is centered in a bold, dark blue font. Underneath the title, there is a form consisting of a text input field with the placeholder text "Enter student name" and a blue button labeled "Add Student". Below the form, there are two student records displayed in a list. Each record is contained within a light pink rounded rectangle. The first record shows the name "tharun" in bold, with "Status: Absent" below it, and two buttons: "Mark Present" (yellow) and "Delete" (red). The second record shows the name "vishal" in bold, with "Status: Absent" below it, and the same "Mark Present" and "Delete" buttons.

Student Attendance Management

Add Student

vishal

Status: Absent

Mark PresentDelete

```
import { useState, useEffect } from 'react';
import axios from 'axios';
import './App.css';

const API_URL = 'http://localhost:5000/api/attendance';

function App() {
  const [attendance, setAttendance] = useState([]);
  const [newStudent, setNewStudent] = useState('');

  // Fetch the attendance list
  const fetchAttendance = async () => {
    try {
      const response = await axios.get(API_URL);
      setAttendance(response.data);
    } catch (error) {
      console.error("Error fetching attendance:", error);
    }
  };
}
```

```

useEffect(() => {
  fetchAttendance();
}, []);

// Add new student (helper function)
const handleAddStudent = async (e) => {
  e.preventDefault();
  if (!newStudent.trim()) return;
  try {
    await axios.post(API_URL, { studentName: newStudent, status:
'Absent' });
    setNewStudent('');
    fetchAttendance();
  } catch (error) {
    console.error("Error adding student:", error);
  }
};

// Update attendance status
const handleUpdateStatus = async (id, currentStatus) => {
  const newStatus = currentStatus === 'Present' ? 'Absent' : 'Present';
  try {
    await axios.put(`${API_URL}/${id}`, { status: newStatus });
    // Update UI instantly
    setAttendance(attendance.map(record =>
      record._id === id ? { ...record, status: newStatus } : record
    ));
  } catch (error) {
    console.error("Error updating status:", error);
  }
};

// Delete attendance record
const handleDelete = async (id) => {
  try {
    await axios.delete(`${API_URL}/${id}`);
    // Update UI instantly
    setAttendance(attendance.filter(record => record._id !== id));
  } catch (error) {

```

```

        console.error("Error deleting record:", error);
    }
};

return (
    <div className="container">
        <h1>Student Attendance Management</h1>

        <form onSubmit={handleAddStudent} className="add-form">
            <input
                type="text"
                value={newStudent}
                onChange={(e) => setNewStudent(e.target.value)}
                placeholder="Enter student name"
                required
            />
            <button type="submit">Add Student</button>
        </form>

        <div className="attendance-list">
            {attendance.map((record) => (
                <div key={record._id} className={`record-card
${record.status.toLowerCase()} `}>
                    <div className="info">
                        <h3>{record.studentName}</h3>
                        <p>Status: <strong>{record.status}</strong></p>
                    </div>
                    <div className="actions">
                        <button
                            className="toggle-btn"
                            onClick={() => handleUpdateStatus(record._id,
record.status)}
                        >
                            Mark {record.status === 'Present' ? 'Absent' : 'Present'}
                        </button>
                        <button
                            className="delete-btn"
                            onClick={() => handleDelete(record._id)}
                        >
                            Delete
                    </div>
                </div>
            ))}
        </div>
    </div>
);

```

```

        </button>
      </div>
    </div>
  )))
  {attendance.length === 0 && <p>No attendance records found.</p>}
</div>
</div>
);
}

export default App;

```

```

.container {
  max-width: 800px;
  margin: 0 auto;
  padding: 2rem;
  font-family: 'Inter', sans-serif;
  color: #333;
}

h1 {
  text-align: center;
  color: #2c3e50;
  margin-bottom: 2rem;
}

.add-form {
  display: flex;
  gap: 1rem;
  margin-bottom: 2rem;
  justify-content: center;
}

.add-form input {
  padding: 0.75rem 1rem;
  font-size: 1rem;
  border: 1px solid #ccc;
  border-radius: 8px;
}

```

```
width: 300px;
}

.add-form button {
  padding: 0.75rem 1.5rem;
  font-size: 1rem;
  background-color: #3498db;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  transition: background-color 0.2s;
}

.add-form button:hover {
  background-color: #2980b9;
}

.attendance-list {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.record-card {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1.5rem;
  border-radius: 12px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  transition: transform 0.2s;
}

.record-card:hover {
  transform: translateY(-2px);
}

.record-card.present {
  background-color: #e8f8f5;
}
```

```
border-left: 5px solid #2ecc71;
}
```

```
.record-card.absent {
  background-color: #fdecec;
  border-left: 5px solid #e74c3c;
}
```

```
.info {
  text-align: left;
}
```

```
.info h3 {
  margin: 0 0 0.5rem 0;
  color: #2c3e50;
}
```

```
.info p {
  margin: 0;
  color: #7f8c8d;
}
```

```
.actions {
  display: flex;
  gap: 0.5rem;
}
```

```
.actions button {
  padding: 0.5rem 1rem;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-weight: bold;
  transition: opacity 0.2s;
}
```

```
.actions button:hover {
  opacity: 0.8;
}
```

```
.toggle-btn {
  background-color: #f39c12;
  color: white;
}

.delete-btn {
  background-color: #e74c3c;
  color: white;
}
```

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const app = express();
app.use(express.json());
app.use(cors());

// Connect to MongoDB
const mongoURI =
"mongodb+srv://vishal:KxqrqciYQcFDVYSh@cluster0.kqdehxf.mongodb.net/attendance_db?retryWrites=true&w=majority&appName=Cluster0";
mongoose.connect(mongoURI)
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.log(err));

// Create Mongoose Schema & Model
const attendanceSchema = new mongoose.Schema({
  studentName: { type: String, required: true },
  status: { type: String, enum: ['Present', 'Absent'], default: 'Absent' },
  date: { type: Date, default: Date.now }
});

const Attendance = mongoose.model('Attendance', attendanceSchema);

// GET: Fetch attendance list
```

```
app.get('/api/attendance', async (req, res) => {
  try {
    const records = await Attendance.find().sort({ date: -1 });
    res.json(records);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

// POST: Add new attendance record (for testing/initialization)

```
app.post('/api/attendance', async (req, res) => {
  try {
    const newRecord = new Attendance({
      studentName: req.body.studentName,
      status: req.body.status || 'Absent'
    });
    const savedRecord = await newRecord.save();
    res.status(201).json(savedRecord);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
```

// PUT: Update attendance status

```
app.put('/api/attendance/:id', async (req, res) => {
  try {
    const updatedRecord = await Attendance.findByIdAndUpdate(
      req.params.id,
      { status: req.body.status },
      { new: true }
    );
    res.json(updatedRecord);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
```

// DELETE: Delete attendance record

```
app.delete('/api/attendance/:id', async (req, res) => {
  try {
```

```
        await Attendance.findByIdAndDelete(req.params.id);
        res.json({ message: 'Record deleted' });
    } catch (err) {
        res.status(400).json({ error: err.message });
    }
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```